

Technik Programista

Testowanie i dokumentowanie aplikacji

Porządki po Flasku

Standardy, kody HTTP, CRUD, dokumentacja
kodu

*Angielski w kodzie, PEP 8, docstringi, kody odpowiedzi, metody HTTP,
konwencje commitów*

Klasa 4 | rok szkolny 2026/2027 | System: Windows

Spis treści

Spis treści

Spis treści.....	2
1. Co jest nie tak z naszym kodem?.....	3
1.1 Zasada.....	3
1.2 Ten sam kod po angielsku	4
1.3 Słowniczek na start.....	4
2. Standardy Pythona: PEP 8.....	5
2.1 Nie sprawdzaj tego ręcznie – narzędzia	5
3. Kody odpowiedzi HTTP	6
3.1 Te, które widziałeś i będziesz widzieć	6
3.2 Ciekawostki.....	7
4. CRUD i metody HTTP	9
4.1 Dlaczego u nas wszystko to GET albo POST?.....	9
4.2 Metody HTTP – ściągą	9
5. Dokumentacja kodu.....	11
5.1 Docstring	11
5.2 Komentarz: dlaczego, nie co	12
5.3 Type hints	12
5.4 README – aktualizacja.....	12
6. Konwencje commitów.....	13
7. Lista kontrolna: zanim powiesz „gotowe”	13
8. Pytania kontrolne.....	15
9. Zadania	16
Zadanie 1: Angielski w kodzie	16
Zadanie 2: Ruff	16
Zadanie 3: Docstringi i type hints.....	16
Zadanie 4: Kody odpowiedzi.....	16
Zadanie 5: Bezpieczny GET	16
Zadanie 6: README i tabela tras	16
Zadanie 7: Historia.....	16
Zadanie 8: Lista kontrolna	16

1. Co jest nie tak z naszym kodem?

Wróć do swojego repozytorium z materiałów 04–07 i spójrz na kod krytycznie. Działa? Działa. Ale:

```
class Produkt(db.Model):
    nazwa = db.Column(db.String(100))
    cena = db.Column(db.Float)
    dostepny = db.Column(db.Boolean)

@app.route("/dodaj", methods=["GET", "POST"])
def dodaj():
    ...
```

Polskie słowa w kodzie. Na początku nauki do INF.03 pisałeś, jak było wygodnie – `$wynik`, `$polaczenie`, `funkcja_dodaj()`. Tu też możesz, nikt Cię nie ukarze. Ale zapamiętaj: **kod piszemy po angielsku**.

Dlaczego? Bo kod czytają ludzie i narzędzia, które polskiego nie znają:

- biblioteki, które importujesz, są po angielsku – `Product.query.filter_by(name=...)` czyta się jak zdanie, `Produkt.query.filter_by(nazwa=...)` jak zlepek,
- `dostepny`, `cena`, `zamowienie` – bez polskich znaków to i tak nie jest polski, tylko kalka,
- w zespole może być ktoś z Ukrainy, Indii, Niemiec – i będzie,
- LLM, z którego korzystasz, generuje lepszy kod przy angielskich nazwach, bo na takich się uczył,
- na GitHubie rekruter widzi `def dodaj_produkta()` i wie, że projekt jest szkolny.

1.1 Zasada

Po angielsku	Po polsku (może być)
nazwy zmiennych, funkcji, klas, metod, pól	komentarze w kodzie
nazwy plików, folderów, modułów	docstringi (na razie; w pracy – angielski)
nazwy tabel i kolumn w bazie	teksty dla użytkownika (szablony, komunikaty)
nazwy tras / endpointów (<code>/products</code> , nie <code>/produkty</code>)	README (na razie)
komunikaty commitów (patrz rozdział 6)	treść zadań, dane testowe

1.2 Ten sam kod po angielsku

```
class Product(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    price = db.Column(db.Float, nullable=False)
    is_available = db.Column(db.Boolean, default=True)

    def gross_price(self):
        return round(self.price * 1.23, 2)

@app.route("/products")
def product_list():
    products = Product.query.order_by(Product.name).all()
    return render_template("products/list.html", products=products)

@app.route("/products/add", methods=["GET", "POST"])
def product_add():
    ...
```

💡 *Pole logiczne nazywaj tak, żeby czytało się jak pytanie: `is_available`, `has_discount`, `can_edit`. Wtedy `if product.is_available`: brzmi jak zdanie.*

1.3 Słowniczek na start

Polski	Angielski	Polski	Angielski
produkt / nazwa / cena	product / name / price	dodaj / edytuj / usuń	add (create) / edit (update) / delete
zamówienie / koszyk	order / cart	lista / szczegóły	list / detail
użytkownik / hasło	user / password	szukaj / filtruj / sortuj	search / filter / sort
kategoria / opis	category / description	dostępny / aktywny	is_available / is_active
ilość / liczba	quantity / count	data utworzenia	created_at
ogłoszenie / rezerwacja	listing (ad) / booking (reservation)	wpis / komentarz	post / comment

2. Standardy Pythona: PEP 8

PEP 8 to oficjalny przewodnik stylu Pythona (<https://peps.python.org/pep-0008/>). Nie musisz go znać na pamięć – musisz pisać tak, żeby Twój kod wyglądał jak kod każdego innego pythonisty. Najważniejsze:

Co	Jak	Przykład
zmienne, funkcje, metody	<code>snake_case</code>	<code>product_list</code> , <code>gross_price()</code> , <code>is_available</code>
klasy	<code>PascalCase</code>	<code>Product</code> , <code>OrderItem</code> , <code>ShoppingCart</code>
stałe	<code>UPPER_CASE</code>	<code>VAT_RATE = 0.23</code> , <code>MAX_ITEMS = 100</code>
moduły, pliki	<code>snake_case</code> , krótko	<code>models.py</code> , <code>app.py</code> , <code>test_models.py</code>
wcięcia	4 spacje, nigdy tabulator	
długość linii	do 79 (lub 88–100 w zespole)	
puste linie	2 między funkcjami na poziomie modułu, 1 między metodami	
importy	na górze pliku: standardowe, potem biblioteki, potem własne	<code>import os / from flask</code> <code>import Flask / from models</code> <code>import db</code>
spacje	<code>a = b + c</code> , <code>f(x, y)</code> , <code>d["key"]</code> – nie <code>a=b+c</code> , <code>f(x,y)</code>	
porównanie z None / bool	<code>if x is None</code> , <code>if flag: – nie == None, == True</code>	

2.1 Nie sprawdzaj tego ręcznie – narzędzia

Ruff to formatter i linter w jednym, bardzo szybki. Instalacja w venv:

```

pip install ruff
ruff format .          # popraw formatowanie (wcięcia, spacje,
                        cudzysłowy)
ruff check .           # wypisz problemy: nieużywane importy, złe nazwy,
                        błędy
ruff check . --fix     # napraw to, co da się automatycznie

```

Uruchom na swoim projekcie i zobacz, co wypisze. Większość to nieużywane importy i spacje. Formatter nie zmienia działania kodu – tylko wygląd.

💡 W PyCharm: `Ctrl + Alt + L` formatuje plik według PEP 8. W VS Code: `Shift + Alt + F` (po instalacji rozszerzenia Python). Ale ruff w terminalu działa wszędzie i można go wpiąć w Git (pre-commit) lub CI.

⚠ **PEP 8 mówi też: nazwy po angielsku. Cytat: identyfikatory w bibliotece standardowej muszą być w ASCII i „SHOULD use English words wherever feasible”.**

3. Kody odpowiedzi HTTP

Maszyna (serwer, aplikacja) rozmawia z Tobą w ustalony sposób. Każda odpowiedź HTTP zaczyna się od trzycyfrowego **kodu statusu**. Umiesz go odczytać Ty, umie przeglądarka, umie aplikacja mobilna, umie skrypt testowy. To wspólny język – dlatego nie wymyśla się własnych.

Pełna tabela: https://pl.wikipedia.org/wiki/Kod_odpowiedzi_HTTP – ale w praktyce używa się kilkunastu. Pierwsza cyfra mówi, co się stało:

Klasa	Znaczenie	Kto zawinił
1xx	informacyjne – rzadko widoczne	–
2xx	sukces	nikt
3xx	przekierowanie – idź gdzie indziej	nikt, to instrukcja
4xx	błąd klienta – Ty (przeglądarka, apka) źle zapytałeś	klient
5xx	błąd serwera – Twój kod się wywalił	serwer

3.1 Te, które widziałeś i będziesz widzieć

Kod	Nazwa	Kiedy	Gdzie już to widziałeś
200	OK	udane GET, udany POST bez tworzenia	każda działająca strona
201	Created	POST utworzył zasób (API)	jeszcze nie – w API
204	No Content	sukces, brak treści (np. DELETE w API)	jeszcze nie – w API
301	Moved Permanently	stały nowy adres	–
302	Found	tymczasowe przekierowanie	<code>redirect()</code> po POST
304	Not Modified	przeglądarka ma to w cache	zakładka Network po odświeżeniu
400	Bad Request	żądanie bez sensu (złe dane)	dzielenie przez zero w kalkulatorze
401	Unauthorized	nie jesteś zalogowany	wkrótce – logowanie
403	Forbidden	zalogowany, ale nie masz prawa	trasa <code>/admin</code> z zadania

Kod	Nazwa	Kiedy	Gdzie już to widziałeś
404	Not Found	nie ma takiej trasy / zasobu	<code>abort(404)</code> , <code>get_or_404</code> , literówka w URL
405	Method Not Allowed	trasa jest, ale nie tą metodą	POST bez <code>methods=[...]</code>
422	Unprocessable Entity	dane poprawne składniowo, ale nie przechodzą walidacji	w API
500	Internal Server Error	wyjątek w Twoim kodzie	każdy traceback w debuggerze
502 / 503	Bad Gateway / Service Unavailable	serwer za proxy nie odpowiada / przeciążony	produkcja, nie u nas

💡 *Reguła kciuka: 4xx – popraw żądanie. 5xx – popraw serwer. Jeśli Twoje API zwraca 500 na złe dane od użytkownika, to błąd – powinno być 400 lub 422.*

3.2 Ciekawostki

418 I'm a teapot. Istnieje naprawdę – żart primaaprilisowy z 1998 roku, RFC 2324 (Hyper Text Coffee Pot Control Protocol). Serwer, który jest czajnikiem, odmawia zaparzenia kawy. W 2017 ktoś chciał go usunąć z bibliotek HTTP; internet zaprotestował i został. Nie używaj go w projekcie – ale na rozmowie o pracę może rozbawić.

Boty skanujące. Wystaw aplikację na publiczny adres i zajrzyj po dobie do logu. Zobaczysz żądania, których nie wysłał żaden Twój użytkownik:

```
45.xx.xx.xx - - "GET /wp-login.php HTTP/1.1" 404 -
45.xx.xx.xx - - "GET /.env HTTP/1.1" 404 -
45.xx.xx.xx - - "GET /phpmyadmin/ HTTP/1.1" 404 -
45.xx.xx.xx - - "GET /.git/config HTTP/1.1" 404 -
45.xx.xx.xx - - "POST /xmlrpc.php HTTP/1.1" 404 -
```

To automaty przeczesujące cały internet w poszukiwaniu WordPressa z dziurą, wystawionego `.env` z hasłami, otwartego `.git`. Twoja apka odpowiada 404 i bot idzie dalej. Ale skoro sam decydujesz o kodzie odpowiedzi, możesz się pobawić:

```
SCANNER_PATHS = {"/wp-login.php", "/.env", "/phpmyadmin/", "/.git/config"}

@app.before_request
def troll_scanners():
    if request.path in SCANNER_PATHS:
        app.logger.warning("scanner: %s %s", request.remote_addr,
            request.path)
        return "I'm a teapot", 418
```

Wariacje, które ludzie naprawdę stosują: zwracanie 200 z fałszywą stroną logowania, żeby bot zmarnował czas (**honeypot**); odpowiedź ważąca kilka gigabajtów zer skompresowanych gzipem do paru kilobajtów (**gzip bomb** – bot próbuje rozpakować i pada); celowe opóźnienie odpowiedzi o 30 sekund (**tarpit**).

Cloudflare, gdy wykryje bota AI ignorującego `robots.txt`, serwuje mu labirynt wygenerowanych stron bez wyjścia.

⚠ Zabawa tak – ale w logach zostaje adres, a poważne rzeczy (blokowanie IP, fail2ban, WAF) robi się na poziomie serwera, nie w Flasku. I nigdy nie odpowiadaj botowi niczym, co zdradza, jakiej technologii używasz.

Nagłówek Server. Flask domyślnie wysyła `Server: Werkzeug/3.x Python/3.12`. Bot od razu wie, czego szukać. Sprawdź w zakładce Network → Headers. W produkcji ten nagłówek się usuwa lub podmienia – nie z paranoi, tylko żeby nie ułatwiać.

Kody, których nie ma w standardzie. Nginx ma 499 (klient zerwał połączenie, zanim serwer odpowiedział), Cloudflare ma 520–530 (problemy między nimi a Twoim serwerem), Microsoft IIS ma 440 (sesja wygasła). Jak zobaczysz kod spoza Wikipedii – to prawie na pewno pośrednik, nie Twoja aplikacja.

451 Unavailable For Legal Reasons. Numer nie jest przypadkowy – to „451 stopni Fahrenheita” Bradbury’ego, powieść o paleniu książek. Kod oznacza: treść zablokowana z powodów prawnych (nakaz sądu, cenzura w danym kraju).

Sprawdź sam: <https://httpbin.org/status/418> – serwis, który zwraca dowolny kod, jaki wpiszesz w adres (`/status/500`, `/status/503`, `/status/404`). Napisał go Kenneth Reitz, autor biblioteki `requests`, bo potrzebował czegoś, do czego można strzelać podczas testów. Przydaje się do sprawdzenia, jak Twoja aplikacja (albo Android, albo React) reaguje na 500 czy 503, bez psucia własnego backendu. Inne endpointy: `/get`, `/post`, `/headers` (pokazuje, co wysłała Twoja przeglądarka), `/delay/3` (odpowiada po 3 sekundach – do testów timeoutów). Do szybkiego podglądu kodu jest też <https://http.cat/418> – każdy kod jako zdjęcie kota.

💡 Serwisy tego typu znikają albo stają za WAF-em (<httpstat.us> w 2026 zwraca 502 przez zaporę). Jeśli link nie działa – to samo dostaniesz lokalnie w 5 liniach Flasku: `@app.route("/status/<int:code>")` zwracające `("", code)`. Zresztą `httpbin` to też aplikacja we Flasku.

4. CRUD i metody HTTP

CRUD = Create, Read, Update, Delete – cztery operacje na danych. Każda warstwa ma dla nich własne słowa:

Operacja	SQL	SQLAlchemy	Metoda HTTP	U nas (Flask + HTML)
Create	INSERT	<code>session.add() + commit()</code>	POST	POST <code>/products/add</code>
Read	SELECT	<code>query.all(), query.get()</code>	GET	GET <code>/products</code> , GET <code>/products/5</code>
Update	UPDATE	zmiana atrybutu + <code>commit()</code>	PUT / PATCH	POST <code>/products/5/edit</code>
Delete	DELETE	<code>session.delete() + commit()</code>	DELETE	POST <code>/products/5/delete</code>

4.1 Dlaczego u nas wszystko to GET albo POST?

Bo formularz HTML zna tylko GET i POST. `<form method="delete">` nie istnieje – przeglądarka wyśle GET. Dlatego usuwanie i edycję robimy POST-em na osobny adres (`/products/5/delete`). To jest poprawne i tak działa większość klasycznych aplikacji.

Pełen zestaw metod poznasz przy API (materiały o Django REST i React): tam klient (JavaScript, Android) wysyła prawdziwe PUT, PATCH, DELETE na ten sam adres `/api/products/5`, a metoda mówi, co zrobić.

4.2 Metody HTTP – ściągą

Metoda	Co robi	Ma ciało?	Bezpieczna?	Idempotentna?
GET	pobierz	nie	tak	tak
POST	utwórz / wykonaj akcję	tak	nie	nie
PUT	zastąp cały zasób	tak	nie	tak
PATCH	zmień część zasobu	tak	nie	zwykle
DELETE	usuń	zwykle nie	nie	tak
HEAD	jak GET, ale same nagłówki	nie	tak	tak
OPTIONS	jakie metody obsługujesz? (CORS)	nie	tak	tak

- **Bezpieczna** = nie zmienia danych. GET nigdy nie może niczego usuwać ani dodawać. Robot indeksujący, podgląd linku w Messengerze, prefetch w przeglądarce – wszystkie wysyłają GET bez pytania.

- **Idempotentna** = wysłana 5 razy daje ten sam efekt co raz. **DELETE** `/products/5` × 5 = produkt usunięty raz. **POST** `/products/add` × 5 = 5 produktów. Dlatego po POST robimy redirect – żeby F5 nie wysłało go drugi raz.

⚠ Jeśli masz gdzieś w projekcie link `<a href="/usun/5">` – to jest GET, który usuwa. Popraw na formularz POST. Na kartkówce o to zapytam.

5. Dokumentacja kodu

Dokumentacja to nie „opis projektu w Wordzie na koniec”. To kilka warstw, z których każda ma innego odbiorcę:

Warstwa	Dla kogo	Gdzie	Odpowiada na pytanie
Nazwy	każdy czytający kod	w kodzie	co to jest? (<code>gross_price</code> , nie <code>gp</code>)
Docstring	ktośwołający funkcję / klasę	pod <code>def</code> / <code>class</code>	co robi, co przyjmuje, co zwraca?
Komentarz	ktoś zmieniający ten fragment	<code># ...</code> obok kodu	dlaczego tak, a nie inaczej?
Type hints	edytor, linter, czytający	w sygnaturze	jakiego typu?
README	ktoś, kto pierwszy raz widzi repo	<code>README.md</code>	co to, jak uruchomić?
Dokumentacja API	programista klienta (React, Android)	OpenAPI / Swagger	jakie endpointy? (jeszcze nie)

5.1 Docstring

Docstring to string w pierwszej linii funkcji, klasy lub modułu. Python go przechowuje – `help(Product)` w konsoli wyświetli go, edytor pokaże po najechnaniu. Standard: **PEP 257**.

```
def gross_price(self) -> float:
    """Zwraca cenę brutto (netto + 23% VAT), zaokrągloną do groszy."""
    return round(self.price * VAT_MULTIPLIER, 2)
```

Jednolinijkowy wystarczy, gdy funkcja jest prosta. Dłuższy, gdy są argumenty i przypadki brzegowe (styl Google, najczęstszy):

```
@app.route("/products/<int:product_id>/edit", methods=["GET", "POST"])
def product_edit(product_id: int):
    """Edycja produktu.

    GET - wyświetla formularz wypełniony aktualnymi danymi.
    POST - zapisuje zmiany i przekierowuje na szczegóły produktu.

    Args:
        product_id: id produktu w bazie.

    Returns:
        Wyrenderowany szablon (GET) lub redirect (POST).

    Raises:
        404: gdy produkt o podanym id nie istnieje.
```

```
"""
product = Product.query.get_or_404(product_id)
...
```

Docstring klasy opisuje, co reprezentuje i co jest ważne:

```
class Product(db.Model):
    """Produkt w sklepie.

    Cena przechowywana jako netto; brutto liczy gross_price().
    Produkt niedostępny (is_available=False) nie znika z bazy,
    tylko nie pokazuje się na liście.
    """
```

Docstring modułu – pierwsza linia pliku:

```
"""Modele bazy danych aplikacji sklepu (SQLAlchemy)."""
from flask_sqlalchemy import SQLAlchemy
```

5.2 Komentarz: dlaczego, nie co

```
# ŹŁE – komentarz powtarza kod:
price = price * 1.23 # mnożymy cenę przez 1.23

# DOBRZE – komentarz tłumaczy powód:
# VAT 23% – stawka podstawowa; obniżone stawki na razie nieobsługiwane
price = price * VAT_MULTIPLIER
```

Jeśli kod potrzebuje komentarza „co robi”, to znaczy, że nazwy są złe. Popraw nazwy, komentarz zniknie.

5.3 Type hints

```
def find_products(query: str, only_available: bool = True) -> list[Product]:
    ...
```

Python ich nie sprawdza w czasie działania, ale edytor podpowiada, linter łapie błędy, a czytający od razu wie, czego się spodziewać. Od dziś: każda nowa funkcja z typami argumentów i zwracanej wartości.

5.4 README – aktualizacja

README z materiału 01 to minimum. Dla projektu z bazą i szablonami dochodzi:

```
## Struktura
app.py          - konfiguracja i widoki (kontrolery)
models.py       - modele SQLAlchemy
templates/      - szablony Jinja2
static/         - CSS
```

```
## Trasy
```

Metoda	Adres	Opis
GET	/products	lista produktów
GET	/products/<id>	szczegóły
GET/POST	/products/add	dodawanie
POST	/products/<id>/delete	usuwanie

💡 *Ta tabela tras to załączek dokumentacji API. Gdy dojdą endpointy JSON, wygenerujemy ją automatycznie (Swagger / OpenAPI). Na razie ręcznie – żebyś wiedział, co tam jest.*

6. Konwencje commitów

Komunikat commita to też dokumentacja – historii projektu. Skoro kod jest po angielsku, commity też. Najpopularniejszy standard to **Conventional Commits** (<https://www.conventionalcommits.org>): typ, dwukropek, krótki opis w trybie rozkazującym.

Typ	Kiedy	Przykład
feat	nowa funkcjonalność	feat: add product search by name
fix	naprawa błędu	fix: handle empty price in add form
refactor	zmiana kodu bez zmiany działania	refactor: rename Polish identifiers to English
docs	tylko dokumentacja	docs: add routes table to README
style	formatowanie, bez zmiany logiki	style: apply ruff format
test	testy	test: add Product.gross_price tests
chore	konfiguracja, zależności, porządki	chore: add ruff to requirements

- Krótko: do 50 znaków w pierwszej linii. Bez kropki na końcu.
- Tryb rozkazujący: **add**, nie **added** ani **adding**. Czytaj jako: „ten commit ... *add product search*”.
- Jeśli potrzebujesz więcej – pusta linia i opis w kolejnych liniach (**git commit** bez **-m** otwiera edytor).

Twoja historia z materiałów 04–07 po konwersji mogłaby wyglądać tak:

```
chore: init Flask project with venv and requirements
feat: add text routes and JSON status endpoint
feat: add dynamic routes with int converter and query string
feat: move HTML to Jinja2 templates with base layout
feat: add product model with SQLAlchemy and CRUD views
refactor: rename Polish identifiers to English
docs: add docstrings and routes table to README
```

7. Lista kontrolna: zanim powiesz „gotowe”

Przed każdym **git push**, który ma być oceniony, przejdź przez listę. To jest **Definition of Done** – w firmach jest spisana i obowiązuje wszystkich.

✓	Sprawdzenie
☐	Aplikacja uruchamia się z czystego klonu: <code>pip install -r requirements.txt</code> , <code>python app.py</code>
☐	Nazwy zmiennych, funkcji, klas, tras – po angielsku, zgodnie z PEP 8
☐	<code>ruff check .</code> nie zgłasza błędów, <code>ruff format .</code> nic nie zmienia
☐	Każda funkcja widoku i każdy model ma docstring
☐	Nowe funkcje mają type hints
☐	Żaden GET nie zmienia danych (usuwanie, dodawanie = POST)
☐	Po każdym POST jest redirect
☐	Nieistniejący zasób daje 404, złe dane 400 – nie 500
☐	<code>.gitignore</code> obejmuje <code>venv/</code> , <code>instance/</code> , <code>*.db</code> , <code>__pycache__</code> , <code>.env</code>
☐	README: co to, jak uruchomić, struktura, tabela tras
☐	Commity małe, po angielsku, według konwencji
☐	Umiesz wyjaśnić każdą linię – także tę, którą napisał LLM

8. Pytania kontrolne

Z tego będą pytania na kolejnych zajęciach. Odpowiedz sobie sam, bez zaglądania:

1. Dlaczego kod pisze się po angielsku, skoro aplikacja jest dla polskich użytkowników? Co w projekcie może zostać po polsku?
2. Jak nazwać: klasę reprezentującą zamówienie, funkcję liczącą sumę zamówienia, stałą ze stawką VAT, pole logiczne „czy opłacone”?
3. Wchodzisz na `/products/999`, produktu nie ma. Jaki kod powinien wrócić i jak to zrobić we Flasku?
4. Formularz wysyła POST na trasę bez `methods=["GET", "POST"]`. Jaki kod zobaczysz? Dlaczego?
5. Czym się różni 401 od 403? Podaj przykład z Twojej domeny.
6. Czym się różni 400 od 500? Kto ma to naprawić?
7. Dlaczego po POST robimy redirect? Jaki kod HTTP przy tym leci?
8. Dlaczego usuwanie nie może być linkiem `<a href>`?
9. Które metody HTTP zna formularz HTML? Gdzie użyjesz `PUT` i `DELETE`?
10. Co to znaczy, że `DELETE` jest idempotentne, a `POST` nie?
11. Czym się różni docstring od komentarza? Co powinno być w jednym, a co w drugim?
12. Co daje `type hint`, skoro Python go nie sprawdza?
13. Jak nazwać commit, w którym poprawiłeś literówkę w README? A taki, w którym zmieniłeś nazwy z polskich na angielskie?
14. Wskaż w swoim projekcie: gdzie jest M, V, C. Która warstwa zna SQL? Która zna HTML?

9. Zadania

Zadanie 1: Angielski w kodzie

Przejdź przez cały projekt (`app.py`, `models.py`, szablony, trasy) i zmień polskie identyfikatory na angielskie. Teksty dla użytkownika w szablonach zostają po polsku. Usuń `instance/*.db` (zmieniły się nazwy kolumn) i uruchom ponownie. Commit: `refactor: rename Polish identifiers to English`.

Zadanie 2: Ruff

`pip install ruff`, dopisz do `requirements.txt`. Uruchom `ruff check .` – zapisz w README, ile problemów znalazł i jakie. Napraw (`--fix` + ręcznie). `ruff format ..`. Commit: `style: apply ruff format and fix lint issues`.

Zadanie 3: Docstringi i type hints

Docstring modułu w `app.py` i `models.py`, docstring każdej klasy i każdej funkcji widoku (dla widoków GET+POST – opisz oba przypadki). Type hints w każdej funkcji przyjmującej argumenty. W `flask shell: help(Product)` – zrób zrzut ekranu do README. Commit: `docs: add docstrings and type hints`.

Zadanie 4: Kody odpowiedzi

Sprawdź w zakładce Network i zapisz w README tabelę: adres → metoda → kod, dla co najmniej 6 różnych przypadków (200, 302, 404, 405, 400, 500). Dla 500 celowo wywołaj wyjątek, a potem napraw. Dla 405 – wyślij POST na trasę bez `methods`.

Zadanie 5: Bezpieczny GET

Przejrzyj projekt: czy jakkolwiek GET zmienia dane? Jeśli tak – przerób na POST. Jeśli nie – napisz w README jedno zdanie, dlaczego to ważne.

Zadanie 6: README i tabela tras

README według rozdziału 5.4: struktura, tabela tras (metoda, adres, opis, możliwe kody odpowiedzi). Commit: `docs: add project structure and routes table`.

Zadanie 7: Historia

Od dziś wszystkie commity według Conventional Commits, po angielsku. Na następnych zajęciach pokażesz `git log --oneline` – co najmniej 4 commity z różnymi typami.

Zadanie 8: Lista kontrolna

Przejdź listę z rozdziału 7 punkt po punkcie. Skopiuj ją do README jako sekcję „Definition of Done” i zaznacz, co spełnione. To, co nie – do poprawy przed oceną.